

Object Oriented Programming for Data Processing

11 September 2025

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 15:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement a C++ class to represent compact binaries. These systems are characterized by 8 intrinsic parameters, given by 2 mass parameters and 6 angular momentum parameters¹. We will operate with the dimensionless angular momentum vectors $\vec{\chi}_1 = (\chi_{1x}, \chi_{1y}, \chi_{1z})$ and $\vec{\chi}_2 = (\chi_{2x}, \chi_{2y}, \chi_{2z})$ [where the index 1 or 2 identifies a BH], and the *chirp mass* and *symmetric mass ratio* parameters, which are given by

$$\mathcal{M}_c = \frac{(m_1 m_2)^{3/5}}{(m_1 + m_2)^{1/5}} \quad \text{and} \quad \nu = \frac{m_1 m_2}{(m_1 + m_2)^2}.$$

respectively. The inversion formulae are

$$m_1 = \frac{1}{2} \frac{\mathcal{M}_c}{\nu^{3/5}} (1 + \sqrt{1 - 4\nu}) \quad \text{and} \quad m_2 = \frac{1}{2} \frac{\mathcal{M}_c}{\nu^{3/5}} (1 - \sqrt{1 - 4\nu}).$$

The class must provide:

- a regular constructor that takes $\mathcal{M}_c, \nu, \vec{\chi}_1, \vec{\chi}_2$ as input; this must enforce that $\nu \in (0, 0.25]$, and that $|\vec{\chi}_i| \leq 1$ and $m_i \in [1, 200]M_\odot$ for $i \in \{1, 2\}$ (for simplicity, set $M_\odot = 1$ and remember to document this in your implementation);
- a default constructor that assumes that $m_1 = m_2 = 1.4M_\odot$ and $\vec{\chi}_i = \vec{0}$ for $i \in \{1, 2\}$;
- methods to determine the individual masses m_1 and m_2 , the mass ratio $q = m_2/m_1$, the *effective spin* and *spin precession* parameters

$$\chi_{\text{eff}} = \frac{m_1 \chi_{1z} + m_2 \chi_{2z}}{m_1 + m_2} \quad \text{and} \quad \chi_p = \frac{1}{A_1 m_1^2} \max(A_1 \chi_{1,\perp}, A_2 \chi_{2,\perp}),$$

¹Given a single compact object, i.e., a black hole or a neutron star, with gravitational mass m and dimensionless rotation angular momentum vector $\vec{\chi} = (\chi_x, \chi_y, \chi_z)$, the intuitive choice is to double these parameters; however, this is not the unique possible choice and it is not necessarily the most convenient.

where $A_1 = (2 + 3q/2)$ and $A_2 = [2 + 3/(2q)]$, and $\chi_{i,\perp} = \sqrt{\chi_{i,x}^2 + \chi_{i,y}^2}$;

- setters for $\mathcal{M}_c$, ν , $\vec{\chi}_1$, $\vec{\chi}_2$, m_1 , m_2 ;
- an overloaded version of the `<<` operator.

Part 2 – Python

On August 26, 2025, the LIGO-Virgo-KAGRA Collaboration released a new version of the Gravitational-Wave Transient Catalog, namely **GWTC-4.0**. Use a Python notebook or Python scripts to do the following tasks.

- Use the `gwosc` package to find out how many gravitational-wave events have been officially uncovered so far and how many **GWTC-4.0** adds to the total count.
- Find out the GPS time of event **GW231123_135430**.
- Use the `gwpv` package to plot the Q -transform of event **GW231123_135430** in each detector involved in the observation; try to set the parameters involved in this operation to create informative plots.
- Repeat the two points above for the event **GW230814_230901**.

You may use the examples provided in class as a starting point.